Independent Component Analysis (ICA)

From Theory to the FastICA Algorithm

A Comprehensive Overview

August 13, 2025

Outline

What is Independent Component Analysis?

- Independent component analysis (ICA) is a statistical method for finding a linear representation of **Non-gaussian data** so that the resulting components are as statistically independent as possible.

What is Independent Component Analysis?

- Independent component analysis (ICA) is a statistical method for finding a linear representation of **Non-gaussian data** so that the resulting components are as statistically independent as possible.
- It is a powerful technique for revealing hidden, underlying factors that create the observed data.

What is Independent Component Analysis?

- Independent component analysis (ICA) is a statistical method for finding a linear representation of **Non-gaussian data** so that the resulting components are as statistically independent as possible.
- It is a powerful technique for revealing hidden, underlying factors that create the observed data.
- Essentially, it aims to "unmix" a set of signals that have been linearly combined.

Motivation: The Cocktail Party Problem

- Imagine two microphones at two different places in a room where two people are speaking simultaneously.
- Each microphone records a weighted sum of both speech signals (s_1 and s_2).
- **The Challenge:** How can we recover the original, separate speech signals using only the mixed recordings?

Motivation: The Cocktail Party Problem

- Imagine two microphones at two different places in a room where two people are speaking simultaneously.
- Each microphone records a weighted sum of both speech signals (s_1 and s_2).
- **The Challenge:** How can we recover the original, separate speech signals using only the mixed recordings?
- ICA can solve this by using the assumption that the original speech signals are statistically independent.

Motivation: Other Applications

- **Biomedical Signal Processing (EEG/MEG):**
  - Electrical recordings from the scalp are mixtures of underlying brain activity components.
  - ICA can find these original brain signals, separating them from noise and artifacts like eye blinks or muscle activity.

Motivation: Other Applications

- **Biomedical Signal Processing (EEG/MEG):**
  - Electrical recordings from the scalp are mixtures of underlying brain activity components.
  - ICA can find these original brain signals, separating them from noise and artifacts like eye blinks or muscle activity.
- **Financial Data Analysis:**
  - Finding hidden driving factors in financial time series, such as daily stock returns or cash flow data from multiple stores.

Motivation: Other Applications

- **Biomedical Signal Processing (EEG/MEG):**
  - Electrical recordings from the scalp are mixtures of underlying brain activity components.
  - ICA can find these original brain signals, separating them from noise and artifacts like eye blinks or muscle activity.
- **Financial Data Analysis:**
  - Finding hidden driving factors in financial time series, such as daily stock returns or cash flow data from multiple stores.
- **Image Processing:**
  - Finding independent features of natural images for tasks like compression and denoising.

Mathematical Definition

We treat each mixture (x_j) and each independent component (s_k) as a random variable.

$$x_j = a_{j1}s_1 + a_{j2}s_2 + \dots + a_{jn}s_n$$

In vector-matrix notation, where $\mathbf{x}$ is the vector of observed mixtures, $\mathbf{s}$ is the vector of source signals, and $\mathbf{A}$ is the mixing matrix:

The Generative ICA Model

$$\mathbf{x} = \mathbf{A}\mathbf{s}$$

- $\mathbf{x}$ is known (observed data).
- $\mathbf{A}$ and $\mathbf{s}$ are the unknowns to be estimated.
- The goal is to find the unmixing matrix $\mathbf{W} = \mathbf{A}^{-1}$ to recover the sources: $\mathbf{s} = \mathbf{W}\mathbf{x}$.

Key Assumptions & Ambiguities

Assumptions:

1. Sources s_i are statistically independent.
2. Source distributions are **non-gaussian**.
3. The mixing matrix $\mathbf{A}$ is square.

Ambiguities:

1. We cannot determine the **variances** (energies) of the sources.
2. We cannot determine the **order** of the sources.

Visualizing the Model

If two independent sources have a uniform distribution (a square), their mixture has a distribution on a parallelogram. The edges of the parallelogram are in the directions of the columns of $\mathbf{A}$. This illustrates how mixing induces dependence.

The Principle: "Non-Gaussian is Independent"

- The **Central Limit Theorem** states that the distribution of a sum of independent random variables tends towards a Gaussian distribution.

The Principle: "Non-Gaussian is Independent"

- The **Central Limit Theorem** states that the distribution of a sum of independent random variables tends towards a Gaussian distribution.
- This implies a mixture of variables is always "more Gaussian" than its individual components.

The Principle: "Non-Gaussian is Independent"

- The **Central Limit Theorem** states that the distribution of a sum of independent random variables tends towards a Gaussian distribution.
- This implies a mixture of variables is always "more Gaussian" than its individual components.
- **ICA reverses this idea:** The core principle is to find a linear combination of the observed mixtures ($y = \mathbf{w}^T\mathbf{x}$) that is as **non-gaussian as possible**.

The Principle: "Non-Gaussian is Independent"

- The **Central Limit Theorem** states that the distribution of a sum of independent random variables tends towards a Gaussian distribution.
- This implies a mixture of variables is always "more Gaussian" than its individual components.
- **ICA reverses this idea:** The core principle is to find a linear combination of the observed mixtures ($y = \mathbf{w}^T\mathbf{x}$) that is as **non-gaussian as possible**.
- Why? A projection $y = \mathbf{w}^T\mathbf{x} = (\mathbf{A}^T\mathbf{w})^T\mathbf{s}$ becomes least Gaussian when it equals one of the original nongaussian sources s_i.
- Therefore, maximizing the nongaussianity of the projection finds one of the independent components.

Measure 1: Kurtosis

The classical measure of nongaussianity. For a zero-mean, unit-variance variable y:

$$kurt(y) = \mathbb{E}\{y^4\} - 3(\mathbb{E}\{y^2\})^2 = \mathbb{E}\{y^4\} - 3$$

- For a Gaussian variable, $kurt(y) = 0$.
- **Positive Kurtosis (Super-gaussian):** "Spiky" distribution, e.g., Laplace.
- **Negative Kurtosis (Sub-gaussian):** "Flat" distribution, e.g., Uniform.

Optimization with Kurtosis

For a mixture $y = z_1 s_1 + z_2 s_2$ with a unit-variance constraint $z_1^2 + z_2^2 = 1$, we have $kurt(y) = z_1^4 kurt(s_1) + z_2^4 kurt(s_2)$. The maxima of $|kurt(y)|$ occur when one of the z_i is zero, meaning y equals one of the sources.

Drawback

Kurtosis is very sensitive to outliers and is not a robust measure.

Measure 2: Negentropy

Negentropy is a more robust measure based on information-theoretic entropy.

- **Entropy (H):** Measures the "randomness" or information content of a variable.

$$H(Y) = -\sum_i P(Y = a_i) \log P(Y = a_i)$$

Measure 2: Negentropy

Negentropy is a more robust measure based on information-theoretic entropy.

- **Entropy (H):** Measures the "randomness" or information content of a variable.

$$H(Y) = -\sum_i P(Y = a_i) \log P(Y = a_i)$$

- A fundamental result is that a **Gaussian variable has the largest entropy** among all random variables of equal variance.

Measure 2: Negentropy

Negentropy is a more robust measure based on information-theoretic entropy.

- **Entropy (H):** Measures the "randomness" or information content of a variable.

$$H(Y) = -\sum_i P(Y = a_i) \log P(Y = a_i)$$

- A fundamental result is that a **Gaussian variable has the largest entropy** among all random variables of equal variance.
- **Negentropy (J):** Measures the difference in entropy between a Gaussian variable and our variable y. It is zero for Gaussian variables and always non-negative.

$$J(y) = H(y_{\text{gauss}}) - H(y)$$

Drawback

Negentropy is computationally very difficult as it requires an estimate of the probability density function (pdf).

Measure 3: Approximations of Negentropy

We use approximations to combine the benefits of kurtosis and negentropy.

Approximation 1: Moment-Based

$J(y) \approx \frac{1}{12}\mathbb{E}\{y^3\}^2 + \frac{1}{48}kurt(y)^2$. This still suffers from the non-robustness of kurtosis.

Measure 3: Approximations of Negentropy

We use approximations to combine the benefits of kurtosis and negentropy.

Approximation 1: Moment-Based

$J(y) \approx \frac{1}{12}\mathbb{E}\{y^3\}^2 + \frac{1}{48}kurt(y)^2$. This still suffers from the non-robustness of kurtosis.

Approximation 2: General Form using non-quadratic functions G

$J(y) \approx \sum_{i=1}^{p} k_i[\mathbb{E}\{G_i(y)\} - \mathbb{E}\{G_i(v)\}]^2$.

- Here, v is a standard Gaussian variable.
- Kurtosis is a special case where $G(y) = y^4$.
- By choosing a function G that does not grow too quickly, we get estimators that are much more robust.
- This provides a good compromise: conceptually simple, fast to compute, and statistically robust.

Framework 1: Minimizing Mutual Information

- An approach from information theory to find an unmixing matrix $\mathbf{W}$ that minimizes the **mutual information** between output components.
- Mutual information (I) is the natural measure of dependence between random variables. It can be interpreted as the reduction in code length gained by coding variables together versus separately.

Framework 1: Minimizing Mutual Information

- An approach from information theory to find an unmixing matrix $\mathbf{W}$ that minimizes the **mutual information** between output components.
- Mutual information (I) is the natural measure of dependence between random variables. It can be interpreted as the reduction in code length gained by coding variables together versus separately.
- For preprocessed data (uncorrelated, unit-variance), a fundamental relationship holds:

$$I(y_1, ..., y_n) = C - \sum_i J(y_i)$$

where J is negentropy and C is a constant.

Framework 1: Minimizing Mutual Information

- An approach from information theory to find an unmixing matrix $\mathbf{W}$ that minimizes the **mutual information** between output components.
- Mutual information (I) is the natural measure of dependence between random variables. It can be interpreted as the reduction in code length gained by coding variables together versus separately.
- For preprocessed data (uncorrelated, unit-variance), a fundamental relationship holds:

$$I(y_1, ..., y_n) = C - \sum_i J(y_i)$$

  where J is negentropy and C is a constant.
- This equation shows that **minimizing mutual information is equivalent to maximizing the sum of the negentropies** ($J(y_i)$), which is our goal of maximizing nongaussianity.

Framework 2: Maximum Likelihood Estimation (MLE)

- A popular, statistics-based approach where we find the unmixing matrix $\mathbf{W}$ that maximizes the likelihood of observing the data.
- The log-likelihood L is given by:

$$L = \sum_{t=1}^{T} \sum_{i=1}^{n} \log f_i(\mathbf{w}_i^T \mathbf{x}(t)) + T \log |\det \mathbf{W}|$$

where f_i are the pdfs of the sources s_i.

Framework 2: Maximum Likelihood Estimation (MLE)

- A popular, statistics-based approach where we find the unmixing matrix $\mathbf{W}$ that maximizes the likelihood of observing the data.
- The log-likelihood L is given by:

$$L = \sum_{t=1}^{T} \sum_{i=1}^{n} \log f_i(\mathbf{w}_i^T \mathbf{x}(t)) + T \log |\det \mathbf{W}|$$

  where f_i are the pdfs of the sources s_i.
- **The Infomax Principle:** A related concept that maximizes the output entropy of a neural network. It is equivalent to MLE if the network's nonlinearity is the cumulative distribution function (CDF) of the source density.

Framework 2: Maximum Likelihood Estimation (MLE)

- A popular, statistics-based approach where we find the unmixing matrix $\mathbf{W}$ that maximizes the likelihood of observing the data.
- The log-likelihood L is given by:

$$L = \sum_{t=1}^{T} \sum_{i=1}^{n} \log f_i(\mathbf{w}_i^T \mathbf{x}(t)) + T \log |\det \mathbf{W}|$$

  where f_i are the pdfs of the sources s_i.
- **The Infomax Principle:** A related concept that maximizes the output entropy of a neural network. It is equivalent to MLE if the network's nonlinearity is the cumulative distribution function (CDF) of the source density.
- **Connection:** Maximizing likelihood is essentially equivalent to minimizing mutual information, as the expected log-likelihood can be expressed in terms of the negative of mutual information.

Practical Pitfall

MLE requires a correct assumption about the source distributions. An incorrect assumption can lead to completely wrong results.

Preprocessing for ICA

Step 1: Centering

The most basic step is to make the data zero-mean by subtracting its mean vector: $\mathbf{x}_{\text{centered}} = \mathbf{x} - \mathbb{E}\{\mathbf{x}\}$.

Preprocessing for ICA

Step 1: Centering

The most basic step is to make the data zero-mean by subtracting its mean vector: $\mathbf{x}_{\text{centered}} = \mathbf{x} - \mathbb{E}\{\mathbf{x}\}$.

Step 2: Whitening

A crucial step that transforms the data vector $\mathbf{x}$ so its components are uncorrelated and have unit variance ($E\{\tilde{\mathbf{x}}\tilde{\mathbf{x}}^T\} = \mathbf{I}$).

- **Benefit:** Whitening reduces the complexity of the problem. It transforms the mixing matrix $\mathbf{A}$ into a new **orthogonal** (rotation) matrix $\tilde{\mathbf{A}}$.
- This reduces the number of parameters to estimate from n^2 to $n(n-1)/2$, effectively "solving half the problem".

The FastICA Algorithm: One Unit

This algorithm finds a single independent component by finding a direction $\mathbf{w}$ where the projection $\mathbf{w}^T\mathbf{x}$ maximizes nongaussianity.

- It uses a **fixed-point iteration scheme**, which is much faster (cubic convergence) than gradient-based methods (linear convergence).
- It assumes data has been preprocessed (centered and whitened).

The FastICA Iteration

1. Choose an initial random weight vector $\mathbf{w}$.
2. Update $\mathbf{w}$: $\mathbf{w}^+ \leftarrow \mathbb{E}\{\mathbf{x}g(\mathbf{w}^T\mathbf{x})\} - \mathbb{E}\{g'(\mathbf{w}^T\mathbf{x})\}\mathbf{w}$.
3. Normalize: $\mathbf{w} \leftarrow \mathbf{w}^+/||\mathbf{w}^+||$.
4. If not converged, go back to step 2.

Here, g is the derivative of a non-quadratic function G, e.g., $g_1(u) = \tanh(a_1 u)$ or $g_2(u) = u\exp(-u^2/2)$.

FastICA for Several Units

To find multiple components, we must decorrelate the outputs to prevent vectors from converging to the same solution.

a. Deflationary Method (One-by-One)

This finds components sequentially. After finding p vectors $(w_1, ..., w_p)$, we find w_{p+1} and after each iteration, subtract the projections of the previously found vectors:

1. $\mathbf{w}_{p+1} \leftarrow \mathbf{w}_{p+1} - \sum_{j=1}^{p} (\mathbf{w}_{p+1}^T \mathbf{w}_j)\mathbf{w}_j$.
2. Renormalize $\mathbf{w}_{p+1}$.

This is similar to the Gram-Schmidt process.

FastICA for Several Units

To find multiple components, we must decorrelate the outputs to prevent vectors from converging to the same solution.

a. Deflationary Method (One-by-One)

This finds components sequentially. After finding p vectors $(w_1, ..., w_p)$, we find w_{p+1} and after each iteration, subtract the projections of the previously found vectors:

1. $\mathbf{w}_{p+1} \leftarrow \mathbf{w}_{p+1} - \sum_{j=1}^{p} (\mathbf{w}_{p+1}^T \mathbf{w}_j)\mathbf{w}_j$.
2. Renormalize $\mathbf{w}_{p+1}$.

This is similar to the Gram-Schmidt process.

b. Symmetric Method (In Parallel)

This method updates all weight vectors simultaneously and is preferred when no component should be prioritized. After each iteration, the entire weight matrix $\mathbf{W}$ is orthogonalized at once.

Properties of the FastICA Algorithm

FastICA has a number of desirable properties compared to other methods:

- **Fast Convergence:** Convergence is cubic (or at least quadratic), which is much faster than the linear convergence of gradient-based algorithms.

Properties of the FastICA Algorithm

FastICA has a number of desirable properties compared to other methods:

- **Fast Convergence:** Convergence is cubic (or at least quadratic), which is much faster than the linear convergence of gradient-based algorithms.
- **Easy to Use:** There are no step size parameters to choose, unlike in gradient descent methods.

Properties of the FastICA Algorithm

FastICA has a number of desirable properties compared to other methods:

- **Fast Convergence:** Convergence is cubic (or at least quadratic), which is much faster than the linear convergence of gradient-based algorithms.
- **Easy to Use:** There are no step size parameters to choose, unlike in gradient descent methods.
- **Versatile:** Finds components of any non-Gaussian distribution using a variety of nonlinearities, without needing an exact estimate of the pdf.

Properties of the FastICA Algorithm

FastICA has a number of desirable properties compared to other methods:

- **Fast Convergence:** Convergence is cubic (or at least quadratic), which is much faster than the linear convergence of gradient-based algorithms.
- **Easy to Use:** There are no step size parameters to choose, unlike in gradient descent methods.
- **Versatile:** Finds components of any non-Gaussian distribution using a variety of nonlinearities, without needing an exact estimate of the pdf.
- **One-by-one Estimation:** The deflationary approach allows for estimating components one at a time, which is useful for exploratory data analysis.

Conclusion

- ICA is a powerful statistical tool for separating linearly mixed signals into their underlying independent, nongaussian sources.
- The core principle is the maximization of nongaussianity, which can be achieved by optimizing measures like kurtosis or negentropy.
- Frameworks like minimizing mutual information or maximizing likelihood provide strong theoretical justification and are closely related.
- Preprocessing via centering and whitening is crucial for simplifying the problem.
- The FastICA algorithm provides a computationally efficient, robust, and easy-to-use method for performing ICA estimation.